

DepoMetrik Akaryakıt Kampanya Karar Motoru: Kapsamlı Bulut Mimarisi ve Veri Mühendisliği Raporu

Yönetici Özeti ve Mimari Vizyon

Akaryakıt ve tüketim takip platformu ekosistemlerinde kullanıcı sadakatini ve finansal faydayı en üst düzeye çıkarmanın temel yolu, araç telemetri verileri ile finansal harcama verilerini akıllı algoritmalarla birleştirmektir. Çevrimdışı öncelikli (offline-first) mimari üzerine inşa edilen mobil araç takip uygulamasının dördüncü fazı kapsamında geliştirilmesi planlanan "Banka Akaryakıt Kampanya Karar Motoru", kullanıcıların finansal harcama alışkanlıkları ile banka kampanyalarını optimize ederek en yüksek ekonomik faydayı sağlamayı amaçlayan sofistike ve dağıtık bir sistem tasarımı gerektirmektedir. Türkiye pazarındaki ana akım finansal kuruluşların sunduğu dinamik, karmaşık ve sürekli değişen akaryakıt kampanyalarının geleneksel yöntemlerle manuel olarak izlenmesi, sürdürülebilir bir mühendislik yaklaşımı değildir. Bu doğrultuda, internet üzerindeki yapılandırılmamış (unstructured) kampanya verilerini otonom bir şekilde tarayacak, yapay zeka (LLM) tabanlı ayrıştırma modelleriyle yapılandırılmış (structured) JSON şemalarına dönüştürecek ve merkezi bulut veritabanı (Supabase PostgreSQL) üzerinden yerel mobil cihazlara (Drift ORM) gerçek zamanlı senkronize edecek uçtan uca bir veri hattı (data pipeline) tasarlanmıştır.

Mimari vizyon, kullanıcının mobil arayüzde kampanya listesini açtığı anda, arka planda herhangi bir gecikme yaşamadan, tüm Türkiye pazarındaki güncel kampanyaları "başlangıç-bitiş tarihleri, ödül tutarları ve hedef işlem sayıları" gibi kritik metriklerle birlikte, temiz ve profesyonel bir arayüzde görüntüleyebilmesini temel almaktadır. Kullanıcı, ilgilendiği kampanyanın detaylarına girmek istediğinde ise yerel veritabanında (SQLite tabanlı Drift) halihazırda bulunan detaylı kurallar bütününe anında erişebilmelidir. Bu rapor, söz konusu kampanya toplayıcı (aggregator) sisteminin bulut mimarisini, veri mühendisliği pratiklerini, anti-bot atlatma stratejilerini, yapay zeka entegrasyonlarını, Supabase PostgreSQL otomasyonlarını ve hukuki uyumluluk çerçevelerini derinlemesine incelemekte; sistemin sıfır sunucu maliyeti (serverless), yüksek ölçeklenebilirlik, hata toleransı ve Türkiye Cumhuriyet yasalarına (TTK, FSEK, KVKK) tam uyumluluk ilkeleri üzerine nasıl inşa edileceğini ortaya koymaktadır.

Veri Toplama Stratejilerinin Karşılaştırmalı Analizi

Kampanya verilerinin merkezi sisteme entegrasyonu için endüstri standardı haline gelmiş çeşitli yöntemler bulunmakla birlikte, her birinin teknik uygulanabilirlik, maliyet ve yasal sürdürülebilirlik açısından farklı dinamikleri mevcuttur. Finansal teknolojiler ekosisteminde (FinTech), bankaların sistemlerine doğrudan erişim sağlamak için Açık Bankacılık (ÖHVPS) API'leri kullanılsa da, bu API'ler genellikle kullanıcı hesap hareketleri ve bakiye sorgulama işlemleri ile sınırlıdır ve genel promosyon/kampanya havuzunu dışarıya açan standart bir uç nokta (endpoint) sunmamaktadırlar. Dolayısıyla, verinin kaynağı olarak bankaların halka açık kampanya web sayfaları temel alınmak zorundadır.

Bu noktada mühendislik ekiplerinin önünde üç ana mimari alternatif belirlemektedir. Birinci

alternatif, yapılandırılmış veri sağlayan ticari veri kazıma (scraping) API'lerini kullanmaktır. Bu platformlar, proxy yönetimi ve tarayıcı otomasyonunu kendi bulut altyapılarında çözerek doğrudan temiz veri sunarlar. İkinci alternatif, "Hizmet Olarak Yapay Zeka Kazıyıcıları" (AI Scraper as a Service) olarak adlandırılan, Firecrawl gibi LLM odaklı bulut araçlarını kullanmaktır. Bu araçlar, web sayfalarını alır ve doğrudan LLM dostu Markdown veya JSON formatında geri döndürürler. Üçüncü alternatif ise, tarayıcı otomasyonu ve veri ayrıştırma mantığının tamamen projenin kendi kontrolünde olduğu, Crawl4AI gibi açık kaynaklı asenkron kütüphanelerin sunucusuz (serverless) ortamlarda barındırıldığı iç kaynaklı (in-house) veri hattı mimarisidir. Aşağıdaki tablo, bu üç temel yaklaşımın DepoMetrik projesinin gereksinimleri doğrultusunda kapsamlı bir karşılaştırmasını sunmaktadır.

Değerlendirme Kriteri	Ticari Veri API'leri (Örn: BrightData)	Bulut Tabanlı AI Kazıyıcı (Örn: Firecrawl)	Açık Kaynak Mimari (Örn: Crawl4AI + GitHub Actions)
Maliyet Yapısı	Yüksek (İstek başına veya aylık sabit ücretlendirme, proxy havuzu maliyetleri)	Yüksek (Aylık 16\$ ile 599\$ arası değişen katmanlı fiyatlandırma, kredi sistemi)	Sıfır (GitHub Actions ücretsiz işlem dakikaları ve açık kaynak kod kullanımı)
Ölçeklenebilirlik	Yüksek (Sağlayıcının sunucu kapasitesi ile sınırlı)	Yüksek (Hizmet sağlayıcı tarafından yönetilen bulut altyapısı)	Orta-Yüksek (CI/CD araçlarının eşzamanlı işlem limitlerine bağlı yatay ölçekleme)
Kurulum Zorluğu	Düşük (Sadece API anahtarı entegrasyonu gerektirir)	Düşük (REST API üzerinden JSON/Markdown yanıt alma kolaylığı)	Yüksek (Tarayıcı konfigürasyonu, proxy yönetimi ve anti-bot sistemlerinin manuel inşası)
Esneklik ve Kontrol	Düşük (Sağlayıcının desteklediği şemalar ve sitelerle sınırlı)	Orta (Sadece sayfanın metinsel içeriğine odaklanma, derin DOM müdahalesi kısıtlı)	Çok Yüksek (Gölge DOM düzleştirme, iframe yönetimi, özel JavaScript enjeksiyonu)
Anti-Bot Atlatma	Gelişmiş (Sağlayıcının tescilli atlatma algoritmaları)	Gelişmiş (Arka planda yönetilen IP rotasyonu ve tarayıcı parmak izi gizleme)	Gelişmiş (Undetected Browser adaptörleri ve Stealth Mode eklentileri ile aşamalı stratejiler)

DepoMetrik'in uzun vadeli sürdürülebilirlik hedefleri ve operasyonel harcamaları (OpEx) minimize etme vizyonu göz önüne alındığında, üçüncü alternatif olan açık kaynak mimarinin seçilmesi en rasyonel mühendislik kararıdır. Başlangıçtaki yüksek kurulum zorluğu, sistemin kendi kendini idame ettiren (self-sustaining) yapısı sayesinde zamanla telafi edilecek ve dışa bağımlılığı ortadan kaldıracaktır.

Dinamik Web Kazıma (Web Scraping) ve Anti-Bot Atlatma Stratejileri

Bankaların kampanya web sayfaları genellikle Tek Sayfa Uygulaması (Single Page Application - SPA) mimarileriyle (React, Angular, Vue.js) geliştirilmiş olup, içerikleri dinamik olarak JavaScript aracılığıyla istemci tarafında (client-side) oluşturulmaktadır. Ek olarak, finansal kuruluşlar siber

saldırıları, içerik hırsızlığını ve DDoS girişimlerini engellemek amacıyla Cloudflare, Akamai veya F5 gibi gelişmiş Web Application Firewall (WAF) ve bot koruma sistemleri kullanmaktadır. Bu karmaşık engelleri aşmak için, statik HTML indiren geleneksel HTTP istemcileri yetersiz kalmakta; tam teşekküllü bir tarayıcı motorunun (Chromium) başsız (headless) modda çalıştırılarak JavaScript kodlarının işletilmesi (rendering) zorunlu hale gelmektedir.

Crawl4AI ile Kendi Kendini Onaran (Self-Healing) Tarama Mimarisi

Açık kaynaklı mimari tercihinin merkezinde konumlandırılan Crawl4AI kütüphanesi, asenkron (asyncio) yapısı ve Playwright entegrasyonu sayesinde modern web sitelerinin dinamik doğasıyla kusursuz bir uyum içinde çalışır. Banka web sitelerindeki temel problem, tasarım ekiplerinin sürekli olarak DOM yapısını, CSS sınıf isimlerini (class names) ve HTML etiket hiyerarşisini değiştirmesidir. Belirli bir XPath veya CSS seçicisine (selector) bağlı kalan geleneksel kazıma scriptleri, bu ufak tasarım değişikliklerinde anında kırılarak (brittle) sistemin çökmesine neden olur.

Crawl4AI, bu kırılganlığı aşmak için "kendi kendini onaran" (self-healing) bir paradigma benimser. Araç, ham HTML DOM ağacını almakla kalmaz; reklamları, gezinme menülerini, altbilgileri (footer) ve içeriğe değer katmayan gereksiz bileşenleri sezgisel (heuristic) BM25 algoritmasıyla filtreleyerek uzaklaştırır. Geriye kalan saf içerik, Büyük Dil Modellerinin (LLM) doğrudan ve yüksek doğrulukla işleyebileceği temiz bir Markdown formatına dönüştürülür. Markdown dönüştürme işlemi DOM bağımsızdır; içeriğin görsel sunumundan ziyade semantik ve metinsel anlamına odaklandığı için, web sitesinin tasarımı baştan aşağı değişse dahi, kampanya metni ekranda okunabilir olduğu sürece kazıma süreci kesintisiz işlemeye devam eder. Bu durum, veri hattının bakım maliyetlerini dramatik bir şekilde düşürür.

Stealth Mode ve Undetected Browser Konfigürasyonları

Bankaların güvenlik duvarlarını aşmak için Crawl4AI'nın sunduğu çok katmanlı savunma atlatma mekanizmaları kritik bir rol oynar. Standart bir başsız tarayıcı, JavaScript ortamındaki navigator.webdriver bayrağının aktif olması, tarayıcı eklentilerinin eksikliği, donanım ivmelendirmesi anomalileri veya sahte ekran çözünürlükleri gibi tarayıcı parmak izi (browser fingerprinting) analizleriyle WAF sistemleri tarafından anında tespit edilerek engellenir. Bu tespiti önlemek adına aşamalı bir kaçınma (progressive evasion) stratejisi uygulanmalıdır.

Birinci aşamada, sistem hafif ve hızlı olan standart "Stealth Mode" (Gizlilik Modu) konfigürasyonu ile istek atar. playwright-stealth eklentisi kullanılarak tarayıcı parmak izi manipüle edilir ve otomatikleştirilmiş yazılım belirtileri gizlenir. Eğer bu aşamada sistem "Access Denied", "Security Check" veya "Cloudflare Ray ID" gibi bir blokaj metniyle karşılaşır, ikinci aşama olan "Undetected Browser" (Tespit Edilemeyen Tarayıcı) moduna geçiş yapar.

Undetected Browser adaptörü, sadece JavaScript ortamını manipüle etmekle kalmaz, tarayıcının ikili (binary) çalışma dosyaları seviyesinde derin yamalar (patches) uygulayarak, en sofistike bot tespit algoritmalarını dahi aşabilen, insan davranışlarına eşdeğer bir tarama profili oluşturur. Aşağıdaki yapılandırma tasarımı, bu aşamalı atlatma stratejisinin Python dilindeki asenkron uygulama mantığını detaylandırmaktadır:

```
import asyncio
from crawl4ai import AsyncWebCrawler, BrowserConfig, CrawlerRunConfig
from crawl4ai.async_logger import AsyncLogger
from crawl4ai.browser_adapters import UndetectedAdapter
```

```

async def fetch_bank_campaigns_with_evasion(url: str) -> str:
    # Aşama 1: Performans odaklı Stealth Mode (Gizlilik Modu) ile
    normal tarama denemesi
    browser_config = BrowserConfig(
        enable_stealth=True,
        headless=True,
        viewport_width=1920,
        viewport_height=1080,
        user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
    )

    async with AsyncWebCrawler(config=browser_config) as crawler:
        result = await crawler.arun(url)

        # WAF/Cloudflare veya Bot koruması tetiklendiğinde HTML içinde
        blokaaj işaretçileri aranır
        block_indicators =
        if result.success and not any(indicator in result.html for
indicator in block_indicators):
            return result.markdown.fit_markdown

    # Aşama 2: Başarısızlık durumunda gelişmiş Undetected Browser
    devreye girer
    print(f"[{url}] Standart Stealth Mode engellendi. Undetected
Adapter başlatılıyor...")
    adapter = UndetectedAdapter()
    strategy = AsyncPlaywrightCrawlerStrategy(
        browser_config=browser_config,
        browser_adapter=adapter
    )

    async with AsyncWebCrawler(crawler_strategy=strategy,
config=browser_config) as crawler:
        config = CrawlerRunConfig(
            flatten_shadow_dom=True, # Modern web bileşenlerindeki
gizli DOM yapılarını düzleştir
            word_count_threshold=20 # Anlam bütünlüğü taşımayan çok
kısa metin bloklarını atla
        )
        result = await crawler.arun(url, config=config)
        return result.markdown.fit_markdown

```

Bu kod blogunda görüldüğü üzere, sistem ağ gecikmelerini ve işlem sürelerini optimize etmek için öncelikle hafif atlatma yöntemlerini dener, yalnızca gerektiğinde daha ağır ve sistem kaynağı tüketen tespit edilemeyen tarayıcıyı başlatır. flatten_shadow_dom parametresi, modern web geliştirme pratiklerinde sıkça kullanılan Web Components teknolojisiyle gizlenmiş içeriklerin

(Shadow DOM) açığa çıkarılarak okunabilmesini sağlar.

Doğal Dil İşleme (LLM) ile Yapılandırılmamış Verilerin JSON Şemasına Dönüştürülmesi

Web sayfalarından başarıyla kazınan ve gürültüden arındırılan Markdown formatındaki metinler, doğaları gereği yapılandırılmamış (unstructured) veya yarı yapılandırılmış verilerdir. Bir bankanın kampanya açıklaması "Farklı günlerde ve tek seferde yapılacak 1500 TL ve üzeri 4. akaryakıt alımına 150 TL Bonus verilecektir" şeklinde doğal dil kurallarına göre yazılmıştır. Bu karmaşık ifadelerin, mobil cihazın ve yerel hesaplama motorunun anlayabileceği, matematiksel işlemlere tabi tutulabilecek kesin SQL tabanlı veri modeline (özellikle campaigns tablosundaki target_tx_count, reward_amount gibi sütunlara) dönüştürülmesi gerekmektedir. Geçmiş yıllarda bu tür veri çıkarma işlemleri için Düzenli İfadeler (Regular Expressions - Regex) veya kural tabanlı doğal dil işleme kütüphaneleri kullanılırdı. Ancak bu yöntemler, dildeki anlamsal çeşitliliği, istisnaları ve farklı ifade biçimlerini yakalamada yetersiz kalarak yüksek oranda veri kaybına veya hatalı eşleşmelere neden olmaktadır. Bu nedenle, semantik bağlamı anlayabilen ve mantıksal çıkarım yapabilen Büyük Dil Modellerinin (LLM) sürece entegre edilmesi kaçınılmaz bir teknolojik gereksinimdir.

Gemini API ve Yapılandırılmış Çıktı (Structured Outputs) Kontrolü

Maliyet etkinliği, milisaniyeler seviyesindeki çıkarım hızı ve yüksek bağlam penceresi (context window) kapasitesi göz önüne alındığında, Google tarafından sağlanan Gemini 2.0 Flash veya Gemini 1.5 Flash modelleri, kampanya verisi ayrıştırma işlem hattı için endüstri standartlarının ötesinde bir performans sunar. Ancak LLM'lerin doğasında var olan "halüsinasyon" (gerçekte olmayan verileri üretme) eğilimini ve sistem şemasına uymayan serbest metin (non-compliant) yanıtlar üretmesini engellemek için, modele sıkı sınırlar çizen Yapılandırılmış Çıktı (Structured Outputs / JSON Schema Enforcement) mimarisi kullanılmalıdır.

Gemini API'nin sunduğu yapılandırılmış çıktı özelliği, modelin ağırlıklarını ve olasılık dağılımlarını, yalnızca önceden tanımlanmış bir JSON şemasına uygun karakterleri (token'ları) üretecek şekilde kısıtlar. Bu, sıfır hata toleransı (zero error tolerance) gerektiren DepoMetrik projesinde veri bütünlüğünün matematiksel olarak garanti altına alınması demektir. Python ekosisteminde veri doğrulama için endüstri standardı olan Pydantic kütüphanesi kullanılarak tanımlanacak şemalar, veritabanındaki hedef tablolar ile birebir aynı veri tiplerine sahip olmalıdır.

Aşağıdaki mimari kod bloğu, Gemini API'nin Pydantic şemaları ile nasıl entegre edileceğini ve sıfır çekim (zero-shot) yöntemiyle karmaşık kampanya metinlerinden nasıl veri çekileceğini göstermektedir:

```
import os
from google import genai
from pydantic import BaseModel, Field

# Google GenAI İstemcisinin ortam değişkenlerinden alınan API anahtarı
ile başlatılması
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

# Pydantic Şemaları: Sistem veri modelinin LLM anlayabileceği dildeki
```

izdüşümü

```
class CampaignRule(BaseModel):
```

```
    bank_name: str = Field(description="Kampanyayı düzenleyen bankanın genel kabul görmüş resmi adı. Örn: Garanti BBVA, İş Bankası, Yapı Kredi, Ziraat Bankası, VakıfBank")
```

```
    station_brand: str = Field(description="Kampanyanın geçerli olduğu hedef akaryakıt markası. Belirtilmemişse veya tüm istasyonlarda geçerliyse 'Genel' yazılmalıdır. Örn: Shell, Opet, BP, Petrol Ofisi")
```

```
    target_tx_count: int = Field(description="Ödüle hak kazanmak için yapılması gereken zorunlu hedef işlem sayısı. Örn: 4")
```

```
    min_tx_amount: float = Field(description="Sayılacak her bir işlemin tek seferde olması gereken minimum tutarı (Türk Lirası cinsinden).")
```

```
    reward_amount: float = Field(description="Kampanya şartları sağlandığında verilecek toplam ödül tutarı (Nakit karşılığı TL veya puan karşılığı TL).")
```

```
    is_different_days_required: bool = Field(description="Kampanya şartlarında işlemlerin 'farklı günlerde' yapılma zorunluluğu var mı? Akaryakıt kampanyalarında suistimali önlemek için genellikle bu şart aranır.")
```

```
    expiry_date: str = Field(description="Kampanyanın son geçerlilik tarihi ve saati, YYYY-MM-DD formatında kesin olarak belirtilmelidir.")
```

```
class CampaignExtractionList(BaseModel):
```

```
    campaigns: list
```

```
def extract_campaigns_from_markdown(markdown_content: str) -> str:
```

```
    # Modelin çalışma sınırlarını belirleyen Sistem Yönergesi (System Prompt)
```

```
    prompt_text = """
```

```
    Sen uzman bir finansal veri analistisin. Aşağıda verilen Markdown formatındaki banka kampanya sayfasını detaylıca incele.
```

```
    İçerisindeki kampanya havuzundan sadece ve sadece 'Akaryakıt', 'Benzin', 'Motorin' veya 'LPG' ile ilgili olan kampanyaları tespit et.
```

```
    Gıda, giyim, e-ticaret, turizm gibi diğer harcama kategorilerine ait kampanyaları kesinlikle görmezden gel.
```

```
    Tespit ettiğin her bir akaryakıt kampanyası için belirtilen katı JSON şemasına uygun olarak verileri ayırıştır.
```

```
    Önemli Kısıtlamalar:
```

```
    - Sonuçlar kesinlikle Türkçe karakter uyumlu olmalıdır.
```

```
    - Tarih formatı ISO 8601 standardına (YYYY-MM-DD) mutlak suretle uymalıdır.
```

```
    - Şarta bağlı kademeli ödüller varsa, ulaşılabilecek en yüksek ödül tutarını baz al.
```

```
    Kaynak Metin:
```

```
    """ + markdown_content
```

```
# Gemini API çağrısı, yapılandırılmış çıktı (Structured Output)
kısıtlamasıyla
response = client.models.generate_content(
    model='gemini-2.0-flash-lite',
    contents=prompt_text,
    config={
        'response_mime_type': 'application/json',
        'response_schema': CampaignExtractionList,
        'temperature': 0.1 # Deterministik, yaratıcılıktan uzak ve
öngörülebilir çıktı için düşük ısı (temperature) değeri
    },
)

return response.text
```

Bu süreç, Crawl4AI tarafından filtre edilen bağlamsal metni alır ve doğrudan Supabase'in veritabanı tablolarına INSERT edilmeye hazır, tip güvenli (type-safe) bir JSON veri kümesine dönüştürür. Gemini modelinin geniş bağlam penceresi (örneğin 1 milyon token sınırına kadar), banka sitesindeki tüm içeriğin parçalanmadan tek bir API isteğiyle işlenebilmesine olanak tanır. Olası çok büyük metin yığınları için Crawl4AI kütüphanesi içerisinde yer alan kayan pencereli yığılma (Sliding Window veya Overlapping Window Chunking) stratejileri devreye sokularak hafıza taşmaları ve token aşırımları engellenebilir.

Merkezi Veritabanı Mimarisi, Eşitleme Katmanı ve Supabase Otomasyonları

Ayrıştırılan ve yapılandırılan kampanya verilerinin, sistemi kullanan tüm mobil cihazlarda tutarlı, gerçek zamanlı ve çevrimdışı çalışmayı destekleyecek bir yapıda görünmesini sağlamak adına Supabase PostgreSQL altyapısının titizlikle tasarlanması gerekmektedir. Faz 3 kapsamında projeye entegre edilen PowerSync motoru, PostgreSQL'deki değişiklikleri Drift yerel (SQLite) veritabanına anında yansıtmakta eşsiz bir görev üstlenmektedir. Ancak, arka planda kampanya havuzunu temiz, güncel ve veri çöplüğünden uzak tutmak için güçlü veritabanı şema kısıtlamalarına (constraints) ve otomasyon mekanizmalarına ihtiyaç vardır.

Veritabanı Şeması Tasarımı ve Satır Bazlı Güvenlik (Row-Level Security)

Kampanya verileri yapısal olarak genel kullanıma açık olsa da, sistemin kalbinde bir kampanyanın kullanıcının kişisel profiliyle eşleştirilmesi yatmaktadır. Kullanıcının hangi kampanyaya katıldığı, kampanyanın hedef işlem sayısının (örneğin 4) kaçınıcısında olduğu gibi durum bilgilerinin tutulması için PostgreSQL üzerinde Çoktan Çoğa (Many-to-Many) ilişkiyi simüle eden bir tablo yapısı kurgulanmalıdır.

PostgreSQL Şema Mimarisi:

1. **global_campaigns tablosu:** Veri kazıyıcı otomasyon tarafından tüm bankalardan çekilen kampanyaların ana referans deposudur. Kullanıcılar bu tabloya hiçbir şekilde müdahale

edemez.

- id (UUID - Primary Key)
- bank_name (varchar, Not Null)
- station_brand (varchar, Not Null)
- target_tx_count (int, Not Null)
- min_tx_amount (double precision, Not Null)
- reward_amount (double precision, Not Null)
- is_different_days_required (boolean, Not Null)
- expiry_date (timestamp, Not Null)
- is_active (boolean, default true)
- *Kısıtlama*: UNIQUE(bank_name, station_brand, expiry_date) - Aynı kampanyanın bot tarafından tekrar tekrar eklenmesini önler.

2. **user_campaigns (Mobil taraftaki orijinal adıyla campaigns)**: Kullanıcının bireysel olarak takip ettiği, katılım sağladığı ve ilerlemesini kaydettiği hedef tablosudur.

- campaign_id (UUID - Primary Key)
- user_id (UUID - Foreign Key -> profiles.user_id, Cascade Delete)
- global_campaign_id (UUID - Foreign Key -> global_campaigns.id, Set Null)
- current_tx_count (int, default 0, Check: >= 0)
- status (enum: ACTIVE, COMPLETED, EXPIRED)

Satır Bazlı Güvenlik (Row-Level Security - RLS) politikaları gereği, global_campaigns tablosuna yönelik SELECT yetkisi tüm doğrulanmış (authenticated) kullanıcılara verilirken, veri ekleme (INSERT), güncelleme (UPDATE) ve silme (DELETE) yetkileri yalnızca hizmet anahtarı (Service Role Key) kullanan arka plan kazıma otomasyonuna aittir. user_campaigns tablosunda ise her kullanıcı, PostgreSQL'in auth.uid() fonksiyonunu temel alan RLS kuralları ile yalnızca kendi benzersiz user_id değerine sahip satırları okuyabilir ve modifiye edebilir. Bu izolasyon, veri sızıntılarını mimari düzeyde engeller.

PostgreSQL Tetikleyicileri (Triggers) ve Veri Hijyeni

Milyonlarca satırlık veritabanlarında, süresi dolan kampanyaların manuel olarak pasife çekilmesi veya temizlenmesi yerine, PostgreSQL'in kendi iç süreçlerine (Tetikleyiciler veya Zamanlanmış Görevler) güvenmek en optimize yöntemdir. Belirli bir zamana bağlı işlemler için (örneğin kampanya bitiş tarihi kontrolü), anlık tablo tetikleyicilerinden ziyade pg_cron eklentisi kullanmak sistem kaynaklarını daha verimli kullanır. Sürekli yüksek okuma/yazma trafiği altındaki tablolarda anlık tetikleyiciler işlem (transaction) darboğazlarına (bottlenecks) sebep olabilir.

Supabase ortamında pg_cron uzantısı kullanılarak, her gece saat 00:01'de asenkron olarak çalışacak ve sistemdeki expiry_date tarihi geçmiş kampanyaları pasif (is_active = false) duruma getirecek zamanlanmış bir görev şu SQL mimarisi ile tanımlanır:

```
-- Süresi dolan kampanyaları pasife çeken SQL fonksiyonunun tanımlanması
CREATE OR REPLACE FUNCTION deactivate_expired_campaigns()
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
    -- Global kampanyaların durumunu güncelle
    UPDATE global_campaigns
    SET is_active = false
```



```

WHERE expiry_date < NOW() AND is_active = true;

-- Süresi dolan kampanyalarla ilişkili kullanıcı takiplerini
güncelle
UPDATE user_campaigns
SET status = 'EXPIRED'
FROM global_campaigns
WHERE user_campaigns.global_campaign_id = global_campaigns.id
AND global_campaigns.is_active = false
AND user_campaigns.status = 'ACTIVE';
END;
$$;

-- pg_cron uzantısı ile görevin sunucu tarafında zamanlanması
SELECT cron.schedule(
    'deactivate-campaigns-cron', -- Zamanlanmış Görev Adı
    '1 0 * * *',                -- Cron formatı (Her gün saat
00:01'de çalışır)
    'SELECT deactivate_expired_campaigns();'
);

```

Supabase ekosisteminde, şema değişikliklerini (Tablo silinmesi, sütun eklenmesi gibi DDL komutları) dinleyen Event Triggers (Olay Tetikleyicileri) mekanizması da mevcuttur. Supautils uzantısı, yetkisiz kullanıcıların (non-superuser) bu tetikleyicileri oluşturmasına belirli güvenlik izolasyonları dahilinde izin verir. Ancak kampanya verisinin güncellenmesi veya süresinin dolması bir DDL (Data Definition Language) değil, DML (Data Manipulation Language) işlemi olduğu için Event Triggers mimarisi bu senaryo için teknik olarak uygunsuzdur. Satır bazlı değişimler için her zaman standart AFTER UPDATE tetikleyicileri veya pg_cron kullanılmalıdır.

Edge Functions ve Gerçek Zamanlı Push Bildirim Mimarisi

Banka Kampanya Karar Motorunun uç kullanıcıya (end-user) sağladığı en büyük değer, kampanya hedefine ulaşıldığında gösterilen proaktif geri bildirimdir. Kullanıcı, hedeflediği akaryakıt istasyonunda kredi kartı ile ödeme yaptığı anda; Faz 2 kapsamında geliştirilen OCR motoru veya SMS Parse mekanizması bu harcamayı tamamen çevrimdışı olarak yakalar ve yerel Drift veritabanına kaydeder. Cihaz internete bağlandığı anda PowerSync motoru devreye girer ve bu yeni işlemi Supabase'deki card_transactions tablosuna eşler.

Bu veri akışı esnasında, kullanıcının kampanya hedefine ulaşip ulaşmadığının (current_tx_count == target_tx_count) sunucu tarafında değerlendirilmesi ve hedefe ulaşıldığında kullanıcıya anlık bildirim (Push Notification) gönderilmesi gerekmektedir. Veritabanında meydana gelen olayların dış dünyadaki bildirim servislerine (Apple Push Notification Service - APNs, Firebase Cloud Messaging - FCM veya Expo Push API) bağlanması için Supabase Edge Functions kullanılır. Edge Functions, V8 motoru üzerinde çalışan Deno tabanlı, soğuk başlama (cold-start) süresi son derece düşük, hafif ve sunucusuz (serverless) scriptlerdir.

Bildirim Tetikleme Algoritması (Postgres Trigger -> Database Webhook -> Edge Function)

Bu ardışık tepkime (chain reaction) mimarisi şu adımlardan oluşur:

1. **PostgreSQL DML Tetikleyici:** user_campaigns tablosunda current_tx_count değeri her güncellendiğinde, veritabanı motoru içindeki bir fonksiyon (PL/pgSQL) çalıştırılır.
2. **Hedef Analizi ve Bildirim Kaydı:** Fonksiyon, güncel işlem sayısını hedef işlem sayısı ile karşılaştırır. Eşitlik sağlandığında, notifications tablosuna asenkron bir bildirim satırı (INSERT) ekler.

```
-- Hedef kontrolünü yapan veritabanı fonksiyonu
CREATE OR REPLACE FUNCTION check_campaign_goal_reached()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    target_count int;
    reward float;
    bank varchar;
BEGIN
    -- İlişkili kampanyanın detaylarını global tablodan çekerek
    hafızaya al
    SELECT target_tx_count, reward_amount, bank_name
    INTO target_count, reward, bank
    FROM global_campaigns WHERE id = NEW.global_campaign_id;

    -- Eğer mevcut işlem sayısı tam olarak hedefe ulaştıysa bildirimi
    sıraya al
    IF NEW.current_tx_count = target_count AND OLD.current_tx_count <
    target_count THEN
        INSERT INTO notifications (user_id, title, body, created_at)
        VALUES (
            NEW.user_id,
            'Kampanya Hedefine Ulaşıldı! 🚀',
            bank || ' kampanyasını başarıyla tamamladınız ve ' ||
reward || ' TL ödül kazandınız. Tebrikler!',
            NOW()
        );
        -- Kampanya durumunu tamamlandı olarak işaretle
        NEW.status = 'COMPLETED';
    END IF;
    RETURN NEW;
END;
$$;

-- Güncelleme olayını dinleyen tetikleyici tanımı
CREATE TRIGGER campaign_goal_trigger
BEFORE UPDATE ON user_campaigns
```

```
FOR EACH ROW
EXECUTE FUNCTION check_campaign_goal_reached();
```

1. **Supabase Database Webhook:** notifications tablosuna eklenen yeni satırları dinleyen yerleşik bir veritabanı kancası (Database Webhook) oluşturulur. Bu kanca, yeni satırın içeriğini bir JSON yükü (payload) olarak paketler ve Supabase ortamında barındırılan push isimli Edge Function'a HTTP POST isteği olarak iletir.
2. **Edge Function (Deno TypeScript) İcrası:** Gelen POST isteğini karşılayan Deno scripti, ilgili kullanıcının profiles tablosunda kayıtlı olan cihaz belirtecini (örneğin Expo Push Token veya FCM Token) çeker ve dış API üzerinden bildirimi yollar.

```
[span_52](start_span)[span_52](end_span)[span_54](start_span)[span_54](end_span)// supabase/functions/push/index.ts (Deno tabanlı Serverless Edge Function)
import { serve } from "https://deno.land/std@0.131.0/http/server.ts";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";

// Edge Function ana HTTP sunucu mantığı
serve(async (req) => {
  // Webhook üzerinden gelen JSON formatındaki veri yükünü ayrıştır
  const payload = await req.json();
  const { user_id, title, body } = payload.record;

  // Supabase ortam değişkenlerinden bağlantı bilgilerini al
  const supabaseUrl = Deno.env.get('SUPABASE_URL')!;
  const supabaseServiceKey =
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!;

  // Yüksek yetkili istemciyi (Service Role) başlat
  const supabase = createClient(supabaseUrl, supabaseServiceKey);

  // Kullanıcının anlık aktif Push Token'ını profiller tablosundan sorgula
  const { data: profile } = await supabase
    .from('profiles')
    .select('expo_push_token')
    .eq('user_id', user_id)
    .single();

  if (profile?.expo_push_token) {
    // Expo Push Notification API'sine güvenli POST isteği oluştur
    const expoResponse = await
    fetch('https://exp.host/--/api/v2/push/send', {
      method: 'POST',
      headers: {
        'Accept': 'application/json',
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${Deno.env.get('EXPO_ACCESS_TOKEN')}`
      }
    })
  }
  // Gelişmiş APNs/FCM entegrasyonu için yetkilendirme
```

```

    },
    [span_56] (start_span) [span_56] (end_span)      body: JSON.stringify({
      to: profile.expo_push_token,
      title: title,
      body: body,
      sound: 'default',
      priority: 'high'
    })),
  });

  // İşlem başarılıysa Expo'dan dönen sonucu HTTP 200 ile geri
  bildir
  return new Response(JSON.stringify(await expoResponse.json()), {
    status: 200 });
}

// Token bulunamazsa hatayı sessizce yut ve HTTP 400 döndür
return new Response("Hedef kullanıcıya ait aktif bildirim token'ı
bulunamadı", { status: 400 });
});

```

Bu çok katmanlı mimari, mobil uygulamanın (Flutter client) bataryayı tüketen ve stabiliteyi tehlikeye atan karmaşık bildirim gönderim döngülerini kendi üzerinde barındırmasını engeller. Sunucu tarafında konumlanan Deno betikleri, Supabase'in aylık 500.000 ücretsiz çağrı (invocation) sunan cömert sınırları dahilinde bu ağır ağ işlemlerini 1000 milisaniyenin altındaki cevap süreleriyle çözer.

Operasyonel Sürdürülebilirlik: Sunucusuz (Serverless) Veri Hattı Tasarımı

Web sayfalarının kazınması, DOM bileşenlerinin oluşturulması ve yapay zeka modelleriyle iletişim kurulması süreçleri doğası gereği yüksek rastgele erişimli bellek (RAM) ve işlemci döngüsü (CPU cycles) gerektirir (özellikle tarayıcı işleme - browser rendering aşamalarında). DepoMetrik platformunun ölçeklenme esnasındaki işletme maliyetlerini (OpEx) sıfıra yakınsamak adına, bulutta sürekli açık kalarak kaynak tüketen bir Sanal Makine (Virtual Machine - EC2, DigitalOcean Droplet) yerine, olay güdümlü ve tamamen sunucusuz (serverless) bir mimari kurgulanmıştır. Bu noktada GitHub Actions platformu, zamanlanmış (cron) görevleri çalıştırmak için mükemmel bir Sürekli Entegrasyon (CI) ortamı sağlamaktadır.

GitHub Actions ve Zamanlanmış Görev (Cron Job) Otomasyonu

GitHub Actions, yazılım depolarındaki .github/workflows/ dizini altındaki YAML tanımlama dosyalarını okuyarak izole edilmiş, her seferinde sıfırdan kurulan (ephemeral) Ubuntu Linux sanal makinelerinde komutlar çalıştırır. Python tabanlı Crawl4AI ve Gemini API entegrasyonunu içeren kazıma senaryosunun (script) günlük olarak tetiklenmesi ve verileri Supabase projesine aktarması için GitHub Actions üzerinde aşağıdaki gibi yapılandırılmış bir görev hattı (workflow) kurulur.

Bu serverless mimarının en büyük avantajı, GitHub'ın sağladığı 2000 dakikalık ücretsiz aylık kullanım kotasından faydalanarak projeye hiçbir maddi yük getirmemesidir. Ayrıca, Supabase projelerinin belirli bir süre işlem görmediğinde inaktif (inactive) duruma düşerek duraklatılmasını da her gece veritabanına veri yazarak engellemiş olur.

```
#.github/workflows/bank_campaign_aggregator.yml
```

```
name: Banka Akaryakıt Kampanya Toplayıcı Veri Hattı
```

```
on:
```

```
  schedule:
```

```
    # Görevin her gün gece 03:00'te (UTC zaman dilimi) çalışmasını  
    sağlayan Cron ifadesi
```

```
      - cron: '0 3 * * *'
```

```
  workflow_dispatch: # Test amaçlı manuel tetikleme yetkisi sağlar
```

```
jobs:
```

```
  scrape-parse-and-sync:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Kaynak Kodu Sanal Makineye Klonla  
        uses: actions/checkout@v4
```

```
      - name: Python Çalışma Ortamını Yapılandır  
        uses: actions/setup-python@v5  
        with:
```

```
          python-version: '3.11'
```

```
          cache: 'pip' # Bağımlılık indirme sürelerini optimize eder
```

```
      - name: Python Bağımlılıklarını ve Tarayıcı Motorlarını Yükle  
        run: |
```

```
          python -m pip install --upgrade pip
```

```
          pip install crawl4ai google-genai pydantic supabase
```

```
          # Playwright aracılığıyla Chromium tarayıcı ikili
```

```
dosyalarını (binaries) indir [span_66] (start_span) [span_66] (end_span)
```

```
          python -m playwright install chromium
```

```
      - name: Veri Kazıma, LLM İşleme ve Supabase Senkronizasyon  
Scriptini İcra Et
```

```
        env:
```

```
          # Ortam değişkenleri, GitHub Repository Secrets üzerinden  
güvenli bir şekilde alınır
```

```
          GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
```

```
          SUPABASE_URL: ${ secrets.SUPABASE_URL }
```

```
          SUPABASE_SERVICE_ROLE_KEY: ${ {
```

```
secrets.SUPABASE_SERVICE_ROLE_KEY }
```

```
        run: |
```

```
          python scripts/aggregator_engine.py
```

Bu görev (job) tetiklendiğinde, Ubuntu ortamında sıfırdan ve izole bir Chromium tarayıcı örneği oluşturulur. Önceden tanımlanmış hedef URL listesi (örneğin VakıfBank Worldcard, Garanti Bonus, İş Bankası Maximiles gibi bankaların resmi kampanya sayfaları) sırayla dolaşılır. Herhangi bir bankanın siber güvenlik sisteminin (WAF) ağ limitine (Rate Limiting) takılmamak ve dürüst kullanım ilkelerine riayet etmek için, script içerisinde asyncio.sleep() fonksiyonuna dayalı rastgele gecikmeler (delays) ve asenkron bekleme rutinleri uygulanmalıdır. Çıkarılan yapılandırılmış JSON objeleri doğrudan Supabase REST API'si veya Supabase Python istemcisi üzerinden global_campaigns tablosuna senkronize edilir.

Faz 5 Vizyonuna Geçiş: Akıllı Rota ve Net Tasarruf Motorunun Temelleri

Geliştirilen bu otonom veri hattı yalnızca Faz 4'ün gereksinimlerini karşılamakla kalmaz; projenin bir sonraki aşaması olan Faz 5'teki "Akıllı Rota Karar Motoru" için zorunlu olan veri temelinin (data bedrock) oluşturur. Bir kullanıcının mobil arayüzde bir kampanyayı karlı olarak değerlendirebilmesi için sadece verilecek ödül tutarına bakması ekonomik olarak yanıltıcıdır. Örneğin, Shell istasyonundan 150 TL değerinde puan kazanmak için, kullanıcının o anki konumundan 10 kilometre yol kat ederek hedef istasyona gitmesi gerekebilir. Bu durumda, mobil cihazın işlemcisi, donanım entegrasyonu (BLE OBD-II) ile toplanan obd_readings verilerini ve tüketim algoritmalarını (Kayan Ağırlıklı Ortalama - Rolling Average) kullanarak tamamen cihaz yerelinde şu algoritmik değerlendirmeyi yapacaktır:

1. **Ekstra Yakıt Maliyeti Analizi:** Harcanacak ekstra mesafe (Δ Distance), aracın kişiselleştirilmiş ortalama yakıt tüketim verisi ($\frac{C_{\text{rolling}}}{100}$) ile çarpılır ve güncel (Faz 3'te EPDK servisinden çekilen) istasyon birim yakıt fiyatı (UnitPrice) ile oranlanarak yolun maliyeti hesaplanır.
2. **Net Tasarruf Optimizasyonu:** Kampanyadan elde edilecek brüt ödül tutarından (c_{reward}), hesaplanan ekstra yol maliyeti çıkarılarak "Net Tasarruf" ($\text{Net}_{\text{Benefit}}$) bulunur. Eğer sonuç sıfırdan büyükse, kampanya UI katmanında görsel olarak vurgulanır.

Bu deterministik karar ağacı ve optimizasyon denklemleri, cihazın yerel Drift veritabanında saklanan verilerle hesaplandığı için bulut sunucu maliyetleri asimptotik olarak sıfıra düşer ve kullanıcı mahremiyeti (konum verisi dahil) buluta iletilmeden korunmuş olur.

Veri Kazımanın Hukuki Boyutları, FSEK ve Haksız Rekabet Analizi

Bulut tabanlı bir veri hattı ve toplayıcı mimarisi tasarlarken mühendislik mükemmelliği tek başına yeterli bir başarı kriteri değildir. Geliştirilen otonom veri toplama yazılımlarının hukuki regülasyonlara, özellikle Türkiye Cumhuriyeti yasalarına ve uluslararası veri koruma standartlarına tam uyum sağlaması gerekmektedir. Banka web sitelerinden otomatik veri çekilmesi eylemi; Fikir ve Sanat Eserleri Kanunu (FSEK), Türk Ticaret Kanunu (TTK) ve Kişisel Verilerin Korunması Kanunu (KVKK) ekseninde oldukça hassas dengelere sahiptir ve sistemin her aşaması bu dengeler gözetilerek tasarlanmıştır.

FSEK ve Sui Generis (Kendine Özgü) Veritabanı Koruması

Türk hukuk sisteminde yazılımların veya botların internet üzerinden otomatik veri çekmesini

(web scraping) doğrudan ve ismen düzenleyen spesifik bir kanun maddesi henüz bulunmamaktadır. Ancak internet siteleri, barındırdıkları bilgilerin derleniş şekli ve veritabanı olmaları hasebiyle FSEK kapsamında iki tür koruma şemsiyesinden yararlanabilirler: Birincisi "Eser Koruması"dır; veritabanının seçimi veya düzenlenişi hususunda bariz bir fikri yaratıcılık (intellectual creation) mevcutsa devreye girer. İkincisi ise "Sui Generis (Kendine Özgü) Koruma"dır; içeriğin elde edilmesi, doğrulanması veya sunulması için veri tabanı üreticisi tarafından "nitelik veya nicelik bakımından esaslı bir yatırım" yapılmışsa geçerlidir. DepoMetrik'in bankalardan çektiği kampanyalar, bankaların fikri yaratıcılığından ziyade olağan ticari promosyon faaliyetlerinin herkesçe bilinen standart listeleridir. Hukuki riskin doğduğu nokta, veritabanının "tamamının veya esaslı bir kısmının" otomatik olarak kopyalanması ve sürekli olarak sistemli bir şekilde sömürülmesidir (extraction). Ancak DepoMetrik mimarisi, hedef siteyi toptan kopyalamak yerine, sayfadaki binlerce gıda, teknoloji, giyim, eğitim ve turizm kampanyasını Gemini LLM modeli yardımıyla eleyerek, **"yalnızca akaryakıt sektörüne ait olan"** kısıtlı bilgileri ayıklar. Bu katı veri minimizasyonu yaklaşımı, bankanın bütüncül veritabanının sömürülmesi argümanını zayıflatmakta ve adil kullanım (fair use) prensiplerine yaklaşmaktadır.

Türk Ticaret Kanunu (TTK) Kapsamında Haksız Rekabet Kriterleri

Web kazıma eylemlerinin hukuk davalarına en sık konu olduğu alan, TTK'nın 55. maddesi ve devamında düzenlenen "Haksız Rekabet" hükümleridir. TTK'nın 55/1-c fıkrası durumu açıkça çerçeveledir: *"Başkasına ait pazarlanmaya hazır çalışma ürünlerini teknik çoğaltma yöntemleriyle devralarak yararlanmak"* dürüstlük kuralına aykırı bir davranıştır ve haksız rekabet teşkil eder. Burada hukuki sonucun tayinindeki en kritik nokta, eylemin taraflar arasındaki "rekabet" dinamiklerine etkisidir.

Yargıtay 11. Hukuk Dairesi ve Yargıtay Hukuk Genel Kurulu'nun güncel kararlarında da defaatle vurgulandığı üzere, haksız rekabetin varlığının kabulü için, bir teşebbüsün diğerinin ticari emeği ve yatırımı üzerinden "bedavacı" (free-rider) bir yaklaşımla haksız bir rekabet avantajı sağlaması ve rakibini zarara uğratması gerekmektedir.

- **DepoMetrik'in Hukuki Pozisyonu:** DepoMetrik platformu, veri çektiği bankalarla rakip (competitor) konumunda olan finansal bir kuruluş veya banka değildir. Bankaların söz konusu kampanyaları yayımlamadaki asıl amacı, promosyonların olabildiğince geniş kitlelere ulaşması, müşteri sadakatının artması ve kendi kredi kartlarının işlem hacminin (transaction volume) yükselmesidir. DepoMetrik, kullanıcıları bu kampanyalardan haberdar eden, kampanya koşullarını görünür kılan ve kullanıcıları hedef bankanın kredi kartını kullanmaya yönlendiren bir "katalizör" veya pazarlama kanalı (aggregator) vazifesi görür. Ortada diğerinin pazar payını çalma durumu olmadığı gibi, bankanın kampanyalarının kullanımını artırdığı için ticari bir zarar değil, tam aksine ortak bir fayda (symbiotic benefit) söz konusudur. Rakip olunmaması durumu, TTK bağlamındaki Haksız Rekabet iddialarının zeminini büyük ölçüde ortadan kaldırmaktadır.

Uluslararası İctihatlar, Kullanım Koşulları (ToS) ve Robots.txt Kuralları

Pek çok finansal web sitesi, alt kısımlarında yer alan Kullanım Sözleşmelerinde (Terms of Service - ToS) "sistem üzerindeki verilerin otomatik botlar, örümcekler veya scraping yazılımları ile toplanamayacağı" yönünde tek taraflı beyanlara yer verir. Ancak oturum açılmadan (unauthenticated) erişilebilen, şifre ile korunmayan ve arama motorları tarafından endekslenmek üzere tüm dünyaya açık bırakılan sayfalardaki halka açık verilerin kazınması ile; kullanıcı hesabı

gerektiren, kapalı sistemlerin kazınması arasında hukuk doktrininde çok net bir çizgi bulunmaktadır.

Avrupa hukuku bağlamında yönlendirici bir emsal olan ve Almanya Federal Mahkemesi (BGH) tarafından 1 Kasım 2024 tarihinde karara bağlanan (Dosya No: VI ZR 10/24) son içtihadta, 'Scraping' faaliyetlerinin dijital hizmetlerin inovasyonu ve geliştirilmesi açısından önemi hukuken kabul edilmiş, veri tabanı hakları ile teknolojik gelişim arasında hassas bir çıkar dengesi (balancing of interests) kurulması gerektiği belirtilerek sektöre yol gösterici bir standart getirilmiştir.

DepoMetrik kazıyıcıları sisteme üye girişi yapmaz, bankaların sistemlerini manipüle etmez ve tamamen statik web sayfalarından okuma işlemi yapar. Operasyonel etiği korumak ve dürüstlük ilkesine riayet etmek adına şu adımlar mimariye dahil edilmiştir:

1. Bankanın sunucusunda yer alan robots.txt direktiflerine uygun hareket edilerek, taranması istenmeyen spesifik dizinlere müdahale edilmez.
2. GitHub Actions üzerinde çalışan betiklerde, istekler arasına kodlanmış gecikmeler (Rate Limiting mekanizmaları) eklenerek banka sunucularının bant genişliğine gereksiz yük bindirilmesinden ve hizmetin aksatılmasından (Denial of Service ihlallerinden) özenle kaçınılır.

KVKK ve Kişisel Veri Minimizasyonu Mimarisi

Bir diğer hukuki boyut olan Kişisel Verilerin Korunması Kanunu (KVKK) açısından sistem mimarisi tam bir izolasyon sağlar. Bankaların herkese açık internet sayfalarındaki kampanyalarda gerçek bir kişiyi tanımlayabilecek herhangi bir kişisel veri (ad, TC kimlik numarası, harcama bakiyesi, hesap özeti) kesinlikle bulunmamaktadır.

Bununla birlikte, kullanıcıların bu kampanyalara katılıp katılmadığını hesaplamak için yapılan kredi kartı harcama analizleri (SMS yakalama ve Kredi Kartı PDF Ayırıştırma), projenin 2. Fazında tasarlandığı gibi hiçbir zaman bulut sunucuya gönderilmez. Kredi kartı ekstresi cihazın yerel geçici belleğinde (RAM) ayrıştırılır, sadece akaryakıt firmalarına ait harcama satırları ayıklanarak yerel veritabanına (card_transactions) kaydedilir. Geri kalan finansal geçmiş anında RAM'den yok edilir. Bu yapı, KVKK Madde 4'te belirtilen "Amaçla bağlantılı, sınırlı ve ölçülü olma (veri minimizasyonu)" ilkesini sistemin temel yapıtaşı haline getirerek sıfır veri sızıntısını garanti etmektedir. Kullanıcının plaka veya konum bilgileri de aynı kapsamda işlenmekte ve cihaz dışına çıkarılması katmanlı aydınlatma metinleriyle açık rızaya tabi tutulmaktadır.

Alıntılanan çalışmalar

1. BonusFlaş – Kart / Kampanyalar - Apps on Google Play, <https://play.google.com/store/apps/details?id=com.garanti.bonusapp>
2. Garanti Bonus | Bedavası En Bol Kredi Kartı, <https://www.bonus.com.tr/>
3. Vakıfkart - VakıfBank Worldcard'ın Ayrıcalıklarla Dolu Dünyasını Keşfedin!, <https://www.vakifkart.com.tr/>
4. Maximum Kart Kampanyaları | Maximiles, <https://www.maximiles.com.tr/kampanyalar/maximum-kart-kampanyalari>
5. How does one build a fintech product like plaid or yodlee? - Reddit, https://www.reddit.com/r/fintech/comments/hypdgq/how_does_one_build_a_fintech_product_like_plaid/
6. Leading Web Scraping API & Solutions Provider in the USA, UK & UAE, <https://www.realdataapi.com/>
7. adityaoryza/scraping-api: The Currency Exchange API provides access to currency exchange rate data sourced from the BCA website. It allows users to index and retrieve exchange rate information based on specific dates, symbols, and date ranges. The

data is collected from <https://www.bca.co.id/id/> - GitHub,
<https://github.com/adityaoryza/scraping-api> 8. Best Crawl4AI Alternative - Firecrawl,
<https://www.firecrawl.dev/alternatives/firecrawl-vs-crawl4ai> 9. Firecrawl - The context API to
search, scrape, and interact with the web at scale., <https://www.firecrawl.dev/> 10. Home -
Crawl4AI Documentation (v0.8.x), <https://docs.crawl4ai.com/> 11. Crawl4AI: Open-source LLM
Friendly Web Crawler & Scraper. - GitHub, <https://github.com/unclecode/crawl4ai> 12. Firecrawl
vs. Crawl4AI vs. Spider: The Honest Benchmark,
<https://spider.cloud/blog/firecrawl-vs-crawl4ai-vs-spider-honest-benchmark> 13.
[crawl4ai/docs/examples/stealth_mode_example.py](https://github.com/unclecode/crawl4ai/blob/main/docs/examples/stealth_mode_example.py) at main - GitHub,
https://github.com/unclecode/crawl4ai/blob/main/docs/examples/stealth_mode_example.py 14.
Undetected Browser - Crawl4AI Documentation (v0.8.x),
<https://docs.crawl4ai.com/advanced/undetected-browser/> 15. Overview of Some Important
Advanced Features - Crawl4AI, <https://docs.crawl4ai.com/advanced/advanced-features/> 16.
LLM-Free Strategies - Crawl4AI Documentation (v0.8.x),
<https://docs.crawl4ai.com/extraction/no-llm-strategies/> 17. LLM Strategies - Crawl4AI
Documentation (v0.8.x), <https://docs.crawl4ai.com/extraction/llm-strategies/> 18.
[gemini-samples/examples/gemini-structured-outputs.ipynb](https://github.com/philschmid/gemini-samples/blob/main/examples/gemini-structured-outputs.ipynb) at main - GitHub,
<https://github.com/philschmid/gemini-samples/blob/main/examples/gemini-structured-outputs.ipynb> 19. Structured outputs - generateContent API - Google AI for Developers,
<https://ai.google.dev/gemini-api/docs/structured-output> 20. Structured output | Gemini Enterprise
Agent Platform - Google Cloud Documentation,
[https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/capabilities/control-gen
erated-output](https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/capabilities/control-generated-output) 21. Generate structured output (like JSON and enums) using the Gemini API |
Firebase AI Logic, <https://firebase.google.com/docs/ai-logic/generate-structured-output> 22. How
to consistently output JSON with the Gemini API using controlled generation - Medium,
[https://medium.com/google-cloud/how-to-consistently-output-json-with-the-gemini-api-using-cont
rolled-generation-887220525ae0](https://medium.com/google-cloud/how-to-consistently-output-json-with-the-gemini-api-using-controlled-generation-887220525ae0) 23. Extraction & Chunking Strategies API - Crawl4AI
Documentation, <https://docs.crawl4ai.com/api/strategies/> 24. Postgres Triggers | Supabase
Docs, <https://supabase.com/docs/guides/database/postgres/triggers> 25. High-Traffic &
PostgreSQL Triggers: Performance Concerns? : r/Supabase - Reddit,
[https://www.reddit.com/r/Supabase/comments/1jqce0e/hightraffic_postgresql_triggers_performa
nce/](https://www.reddit.com/r/Supabase/comments/1jqce0e/hightraffic_postgresql_triggers_performance/) 26. Event Triggers | Supabase Docs,
<https://supabase.com/docs/guides/database/postgres/event-triggers> 27. PostgreSQL Event
Triggers without superuser access - Supabase,
<https://supabase.com/blog/event-triggers-wo-superuser> 28. Real-Time Push Notifications with
Supabase Edge Functions and Firebase - Medium,
[https://medium.com/@vignarajj/real-time-push-notifications-with-supabase-edge-functions-and-fi
rebase-581c691c610e](https://medium.com/@vignarajj/real-time-push-notifications-with-supabase-edge-functions-and-firebase-581c691c610e) 29. Build Your Own Push Notification System for Free with Supabase
Edge Functions - Reddit,
[https://www.reddit.com/r/reactnative/comments/1jagsy4/build_your_own_push_notification_syst
em_for_free/](https://www.reddit.com/r/reactnative/comments/1jagsy4/build_your_own_push_notification_system_for_free/) 30. Sending Push Notifications | Supabase Docs,
<https://supabase.com/docs/guides/functions/examples/push-notifications> 31. Using PostgreSQL
triggers to automate processes with Supabase - YouTube,
<https://www.youtube.com/watch?v=0N6M5BBE9AE> 32. best strategy for adding push
notifications? · supabase · Discussion #13930 - GitHub,
<https://github.com/orgs/supabase/discussions/13930> 33. Scheduled Scraper Github Actions
Cron - Tools in Data Science,
<https://tds.s-anand.net/2026-02/docs/labs/week-6/scheduled-scraper-github-actions-cron/> 34.

How to web scrape on Schedule using Github Actions? - Yasoob Khalid,
<https://yasoob.me/posts/github-actions-web-scraper-schedule-tutorial/> 35.
tools-in-data-science-public/scheduled-scraping-with-github-actions.md at main,
<https://github.com/sanand0/tools-in-data-science-public/blob/main/scheduled-scraping-with-github-actions.md> 36. How to Set Up a GitHub Actions Cron Job to Prevent Supabase Inactivity - DEV Community,
<https://dev.to/nasreenkhalid/how-to-set-up-a-github-actions-cron-job-to-prevent-supabase-inactivity-3m6b> 37. Web Scraping/Kazıma ve İnternet Sitelerinin Bundan Korunması,
<https://www.guleryuz.av.tr/tr/news-publications/detail/https-wwwguleryuzavtr-tr-news-publications-detail-web-kazima-web-scraping-ve-internet-sitelerine-yonelik-koruma> 38. Yeni Türk Ticaret Kanunu'nda Haksız Rekabet Hükümleri | Erdem&Erdem,
<https://www.erdem-erdem.av.tr/bilgi-bankasi/yeni-turk-ticaret-kanununda-haksiz-rekabet-hukumleri> 39. WEB SCRAPING EYLEMİNİN HAKSIZ REKABET AÇISINDAN DEĞERLENDİRİLMESİ,
<https://www.goksusafiisik.av.tr/tr/publications/2025-summer-issue/web-scraping-eyleminin-haksiz-rekabet-acisindan-degerlendirilmesi?id=510> 40. Haksız Rekabet Davaları – Güncel Yargıtay Kararları (2024-2025 Örnekleri) - Av. Ayça Bolak,
<https://www.aycabolak.av.tr/haksiz-rekabet-davalari-guncel-yargitay-kararlari-2024-2025-ornekleri/> 41. Rekabet Hukuku - Yargıtay Kararları,
https://karamercanhukuk.com/yargitay-kararlari/?category_title=rekabet-hukuku&category_id=35 42. Yargıtay, Scraping Hukuk Mücadelesinde Kılavuz Karar Belirledi | MTR Legal,
<https://www.mtrlegal.com/tr/yargitay-scraping-hukuk-mucadelesinde-kilavuz-karar-belirledi/>